Bito

GOVERNOR PROPOSAL

PREPARED FOR COMCAST / XFINITY · AUGUST 27, 2026

48% Reduction in Agent Token Costs for Comcast / Xfinity

Inject the right code context up front, then route each request to the right model. Governor, Bito's context-driven model router, manages both factors of your AI bill.

Executive summary

A platform engineering organisation of 2,500 engineers running Kubernetes-based microservices across cloud and on-prem generates substantial AI coding spend the moment agents enter the loop. Each agentic task that lacks codebase context burns tokens on exploration, re-prompting, and throwaway generation before producing anything usable. At that scale, even a modest per-engineer agent workload compounds into seven-figure annual model spend — and the 2025 architectural consolidation, which is actively reshaping service boundaries, makes context-blind agents more expensive, not less.

Governor addresses this at the unit-economics level: the Code Context Engine pre-indexes the actual codebase and injects relevant context before each request, cutting token cost per task by 47% on average. The Model Router then sends each request to the lowest-cost model that clears the quality bar. For the AI Operations Platform being built now, that means agents grounded in real service contracts and Kubernetes topology from day one — without changing the harness, the workflow, or the model contracts the team already uses.

Our solution

Governor is a transparent layer between your engineering harness and your LLMs. You update the base URL in your existing harness to point at Governor — one base-URL swap — and nothing else in your developers' workflow changes. Governor manages both multipliers of your AI bill at once: total token count and price per token.

- **The Code Context Engine cuts token volume.** AI Architect maintains a continuously updated semantic map of your codebases. When an agent gets a task, Governor resolves it to the code that answers it and injects that context up front, so the agent skips most of the file reading and gets to the fix sooner: **47% lower token cost per task** in aggregate, up to 68% on individual tasks.
- **The Model Router cuts price per token.** Once the right code is in the prompt, most requests stop needing frontier-grade reasoning, so each one is routed to the lowest-cost model that clears the quality bar — and frontier models are never pulled off the work that needs them.

Quality doesn't pay for it. In our customer A/B, task success was 100% in both arms; in independent SWE-Bench Pro testing the same context engine lifted task success from **51.9% to 70.1%**.

How it works

Day 1 — drop-in endpoint

Update the base URL in your existing harness (Claude Code, Cursor, Codex, Copilot). Works direct to model providers or in front of your existing gateway, with zero behavioral change for your engineers and no disruption to active sprints.

Week 2 — codebase mapping

AI Architect indexes your repositories into a continuously updated map of services, architectural patterns, and dependencies — deeper than vector search, and inside your security and compliance guardrails. Governor starts injecting that context up front.

Month 2 — instrumented A/B on your own workload

The same work runs with and without Governor. You get a telemetry report showing the exact reduction in reasoning turns, tool calls, task time, and token cost — your benchmark, on your codebase, not ours.

Comcast / Xfinity's numbers

	Annual
Estimated token spend today 57,000 engineers × $1,000/month assumed allowance	$684,000,000
At Governor's published −47% per task bill brought down to $362,520,000	**$321,480,000 saved**
At the −83% measured on a full working session	**$567,720,000 saved**

The allowance is an assumption to make the math concrete — the instrumented A/B below replaces it with your real per-task numbers.

Proof points

- Controlled customer A/B, identical tasks on Claude Code: cost per task **$4.12 → $2.14 (−48%)**, reasoning turns halved (47.6 → 23.6), tool calls −53%, tasks 36% faster. Task success 100% in both arms.
- Full ten-ask working session, five runs per arm, same model and caching: **$6.81 → $1.13 (−83%)**, tokens through the model 5.3M → 1.4M, 10/10 tasks vs 9/10 for the baseline.
- SWE-Bench Pro (independent, The Context Lab): task success **51.9% → 70.1%** with Governor's code context.
- In production: PubMatic (500+ repos, 450 engineers) cut technical design per large epic from **2 weeks to 3 days**; Privado shipped an enterprise SSO project in **5 hours** against a 7–10 day estimate.

Why Bito

Both ends of your AI stack are moving targets: models leapfrog each other every few months and harnesses ship weekly. The one stable place to manage cost, quality, and policy is the layer between them. Model routing saves real money, and before long every vendor will offer a version of it. The deeper savings — reducing the token count itself — require a semantic understanding of your proprietary codebase. That's Bito's foundation, and Governor is where the context engine, the routing, and every future optimization live together.

Next steps

The most direct way to validate this is a controlled A/B benchmark on a representative slice of the platform engineering workload: same tasks, same harness, Governor on one side and the current setup on the other, measuring token cost and task success against your own codebase. We would like to schedule a 30-minute technical kickoff with the AI Operations Platform team to scope that benchmark and agree on success criteria.
